USB Device Activity Monitor

Final Project Report

SDN260S Software Design 2

Group 37



Name	Student Number
Ashwill Visagie	230225934
Kwalunga Ligwa	222385561
Tumisang Mainaakae	230999891
Wande Mchwayi	231003463
Thato Ernest	230383920

Due Date - 08 November 2026

1. Problem Definition and Requirements

1.1 The problem

The USB drives are a common route for data theft, malware infection and accidental data loss. Most of the users have no record of which devices were plugged into their computer when they were trusted. This matters because an unnoticed device can copy data or introduce malware within seconds without a history.

1.2 Our solution

We as a group built a Java tool that detects when USB drives are connected or removed, records the device details and time and flags devices that are not on a trusted list. It let the user review, search, sort and export that history. It runs from a command-line interface and a Swing graphical dashboard. It includes a simulation mode so the system can be tested and demonstrated without physical hardware.

1.3 Functional requirements

ID	Requirement	Main classes
FR1	Detect connection and removal of USB drives	PollingUsbDetector, UsbMonitorService
FR2	Record device ID, label, mount point, file system, size and event time	UsbDrive, UsbEvent, EventLogger
FR3	Flag untrusted devices; mark a device as trusted (GUI)	TrustedDeviceRegistry, MonitorGui
FR4	Keep history between runs	EventRepository (serialization)
FR5	Search (by label or drive) and sort (by time) the history in the GUI	MonitorGui, EventRepository
FR6	Scan a drive's contents recursively and summarize folders, files and space	DriverScanner
FR7	Export history to a CSV report	ReportGenerator
FR8	Provide a CLI and a Swing GUI, plus a simulation mode	CliMenu, MonitorGui, MockUsbDetector, UsbGroup37

1.4 Non-functional requirements

Requirement	How it is met
Reliability	File I/O and scanning errors are caught and reported as messages; the application keeps running
Low resource use	Detection runs on a background thread at a fixed polling interval, not a busy loop
Usability	A simple launcher, a six-button dashboard and a history table
Maintainability and testability	Interfaces decouple components; a mock detector replaces real hardware; the code is organised into four logical layers (data, service, storage, user interface)

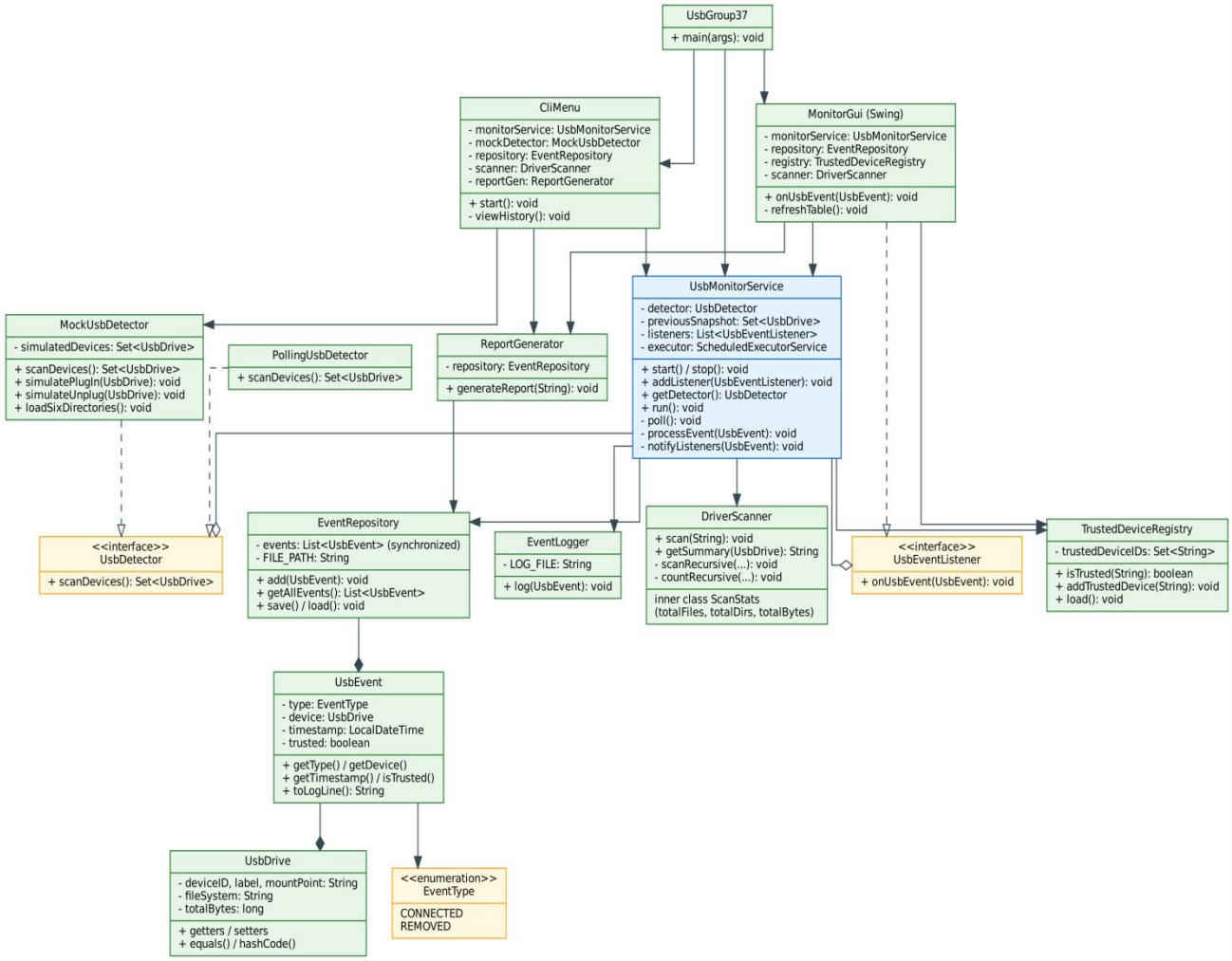
2. System Design and Architecture

2.1 Package structure

All classes are in a single Java package, usbgroup37, but the code is organised into four logical layers, each with a single responsibility.

Layer	Responsibility	Classes
Data model	Data classes	UsbDrive, UsbEvent, EventType
Service	Detection and business logic	UsbMonitorService, UsbDetector, PollingUsbDetector, MockUsbDetector, UsbEventListener, EventLogger, TrustedDeviceRegistry, DriverScanner
Storage	Persistence and reports	EventRepository, ReportGenerator
User interface	User interfaces and launcher	UsbGroup37, CliMenu, MonitorGui

2.2 UML class diagram



2.3 Design principles applied

- **Abstraction and decoupling:** UsbMonitorService depends on the UsbDetector interface, not on a specific detector. PollingUsbDetector reads real drives; MockUsbDetector supplies simulated drives. Either can be swapped in without changing the service.
- **Observer pattern:** UsbMonitorService keeps a list of UsbEventListener objects and notifies each one (notifyListeners) when an event occurs. MonitorGui registers as a listener and refreshes its history table. The service writes each event to the log file (EventLogger) and stores it in the repository (EventRepository) directly, before notifying the listeners.

- **Encapsulation:** model classes keep their fields private and expose them through getters; UsbDrive overrides equals and hashCode so drives can be compared and stored in collections correctly.
- **Separation of concerns:** user interface classes do not detect drives or write the log themselves; they call the monitoring service, the scanner and the storage classes.

2.4 Data flow

1. A ScheduledExecutorService in UsbMonitorService calls poll() every 2 seconds.
2. poll() asks the UsbDetector for the current set of drives (scanDevices()).
3. The service compares this set with the previous snapshot (a Set<UsbDrive>) to find newly connected and removed drives.
4. For each change it creates a UsbEvent, checks TrustedDeviceRegistry to set the trusted flag, and calls processEvent(event).
5. processEvent appends a line to the log file (EventLogger), stores the event in EventRepository (which also saves it to disk) and calls notifyListeners(event), so every registered UsbEventListener, such as MonitorGui, receives onUsbEvent(event) and refreshes its table.
6. On a CONNECTED event the service also starts a background thread that calls DriverScanner.getSummary(drive) and prints the result.
7. The user can view history, scan a drive or folder (DriverScanner) or export a CSV (ReportGenerator) from the CLI or GUI.

3. Implementation Details

3.1 Class descriptions

Class	Description
UsbDrive	Represents a drive: deviceID, label, mountPoint, fileSystem, totalBytes. Serializable; overrides equals and hashCode (drives are compared by deviceID).
UsbEvent	One event: EventType (CONNECTED or REMOVED), the drive, a LocalDateTime timestamp and a trusted flag. Serializable. toLogLine() formats it for the log.
EventType	Enum with CONNECTED and REMOVED.
UsbDetector	Interface with scanDevices() returning the current set of drives.
PollingUsbDetector	Implements UsbDetector by listing the file-system roots, skipping local, fixed and network drives (using the Windows drive-type description) and converting the

Class	Description
	rest to UsbDrive objects.
MockUsbDetector	Implements UsbDetector with an editable set of drives (simulatePlugIn, simulateUnplug, loadSixDirectories) for testing and Simulation Mode.
UsbMonitorService	Core service: start and stop (a ScheduledExecutorService polls every 2 seconds), addListener, getDetector, poll, processEvent and notifyListeners. Compares the current drives with the previous snapshot to find connections and removals.
UsbEventListener	Interface with onUsbEvent(UsbEvent). Implemented by MonitorGui.
EventLogger	Appends one line per event to usb_log.txt (log). Called directly by UsbMonitorService.
TrustedDeviceRegistry	Holds a Set<String> of trusted device IDs with isTrusted, addTrustedDevice, save and load (trusted.ser).
DriverScanner	Recursively walks a folder: scan prints the tree and totals; getSummary(UsbDrive) returns folder, file and size totals (simulated figures for MOCK drives). Uses an inner ScanStats class.
EventRepository	Stores events in a synchronized list: add, getAllEvents (returns a copy), save and load using serialization (events.ser).
ReportGenerator	Exports the history to a CSV file (generateReport), for example usb_report.csv.
CliMenu	Console menu: start and stop monitoring, view history, generate a CSV report, simulate a plug-in, scan a folder and exit.
MonitorGui	Swing dashboard and history screen: search by label or drive, sort by time, mark a device as trusted, and click a row to see the drive summary.
UsbGroup37	Launcher (main): loads saved history and trusted devices, then offers CLI, GUI with real hardware, or GUI in Simulation Mode.

3.2 Algorithms and data structures

Item	Where	Why	Complexity (n = number of items)
Set diff of drive sets	UsbMonitorService.poll	Compare current and previous drive sets to find connects and removals	O(d) per poll with hash-based lookup, d = drives
HashSet of IDs	TrustedDeviceRegistry	Fast trust check and no duplicates	O(1) average lookup
List of events	EventRepository (synchronized list)	Keeps chronological history	O(1) add in memory; save() rewrites the file, O(n)
Search with a stream filter	MonitorGui.refreshTable	Filter events by label or mount point	O(n)
Sort with Comparator	MonitorGui.refreshTable	Sort events by time, newest or oldest first	O(n log n)
Recursion	DriverScanner.scanRecursive / countRecursive	Traverse nested folders to any depth	O(f), f = files and folders visited
HashSet snapshot	UsbMonitorService	Remember the previous state of the drives to detect changes	O(1) average

3.3 Advanced Java features

- **Multithreading:** a ScheduledExecutorService runs detection in the background so the interface stays responsive; short-lived threads run drive scans; the event list is synchronized so the monitor thread and the interface can share it safely.
- **Object serialization:** the event history is saved and reloaded, so it survives restarts (Figure 2 shows events from several earlier sessions).
- **Generic collections:** List<UsbEvent>, Set<String> and Set<UsbDrive>.
- **Lambdas and functional interfaces:** lambdas handle button actions and the stream filter used for search, and Comparator.comparing is used for sorting.
- **Interfaces and polymorphism:** UsbDetector and UsbEventListener.
- **File handling:** the log file, the serialized data file and the CSV report.

3.4 Error handling and logging

File, scan and report operations are wrapped in exception handling and report problems as messages. For example, scanning a path that does not exist produces the message "Error: The path ... does not exist" and the program continues (Figure 4). Every event is appended to the log file by EventLogger, and the GUI confirms report generation to the user (Figure 3).

4. User Guide

Requirements: Java JDK and Apache NetBeans or VS Code; a computer that can mount USB drives.

Running the application:

1. Open the project in NetBeans or VS Code and run `UsbGroup37`.
2. Choose a mode: 1 for the command line, 2 for the GUI with real hardware, or 3 for the GUI in Simulation Mode.
3. Select **Start Background Monitoring**, then plug in or remove a USB drive. In Simulation Mode, use **Simulate USB Plug-in**.
4. Select **View Current & Past History** to see the table. Use the search box and sort button to filter and order it, select a row and click **Mark as Trusted** to trust that device, or click a row to see the drive's files and space summary.
5. Select **Generate CSV Report** to export the history to `usb_report.csv`.
6. Select **Stop Monitoring** and **Exit** when finished.

Troubleshooting: if no events appear, check that monitoring has been started and that the drive has a mount point. Use Simulation Mode to test without hardware.

5. Testing and Evaluation

5.1 Approach

We tested each feature with normal and edge inputs, using real USB drives where possible and the MockUsbDetector for repeatable cases. The tests below are those verified from the running system; the figures referred to are in Section 5.3.

5.2 Test cases and results

ID	Feature	Input / action	Type	Expected	Result
T1	Detection	Plug in a real flash drive	Normal	CONNECTED event with time, drive and label	Pass (Figure 2: MyFlashDrive on E:)
T2	Detection	Remove the same drive	Normal	REMOVED event	Pass (Figure 2: REMOVED for E:\

ID	Feature	Input / action	Type	Expected	Result
					seconds later)
T3	Trust flagging	Plug in an untrusted drive	Normal	Status shows UNTRUSTED	Pass (Figure 2)
T4	Persistenc e	Close and reopen after earlier sessions	Normal	Old events reloaded	Pass (Figure 2 shows events from 17 and 18 October)
T5	Simulation	Start Simulation Mode	Normal	Six mock drives plugged in and monitoring starts	Pass (Figure 4)
T6	Scanner	Scan a path that does not exist	Edge	Error message, no crash	Pass (Figure 4)
T7	Report	Generate CSV from the GUI	Normal	usb_report.csv created and confirmed	Pass (Figure 3)
T8	Summary	Click a history row	Normal	Files and space summary shown	(panel visible in Figure 2)
T9	Search and sort	Search the history by label or drive and sort it by time	Normal	Correct matches and order	**[CONFIRM]**
T10	Search	Search with no matches	Edge	Empty result, no exception	**[CONFIRM]**
T11	Trust	Select a row and click Mark as Trusted	Normal	Status changes to TRUSTED and stays after a restart	**[CONFIRM]**
T12	Reports	Export with no events	Edge	Header-only CSV created, no exception	**[CONFIRM]**

5.3 Evidence

Figures 1 to 4 show the running system.

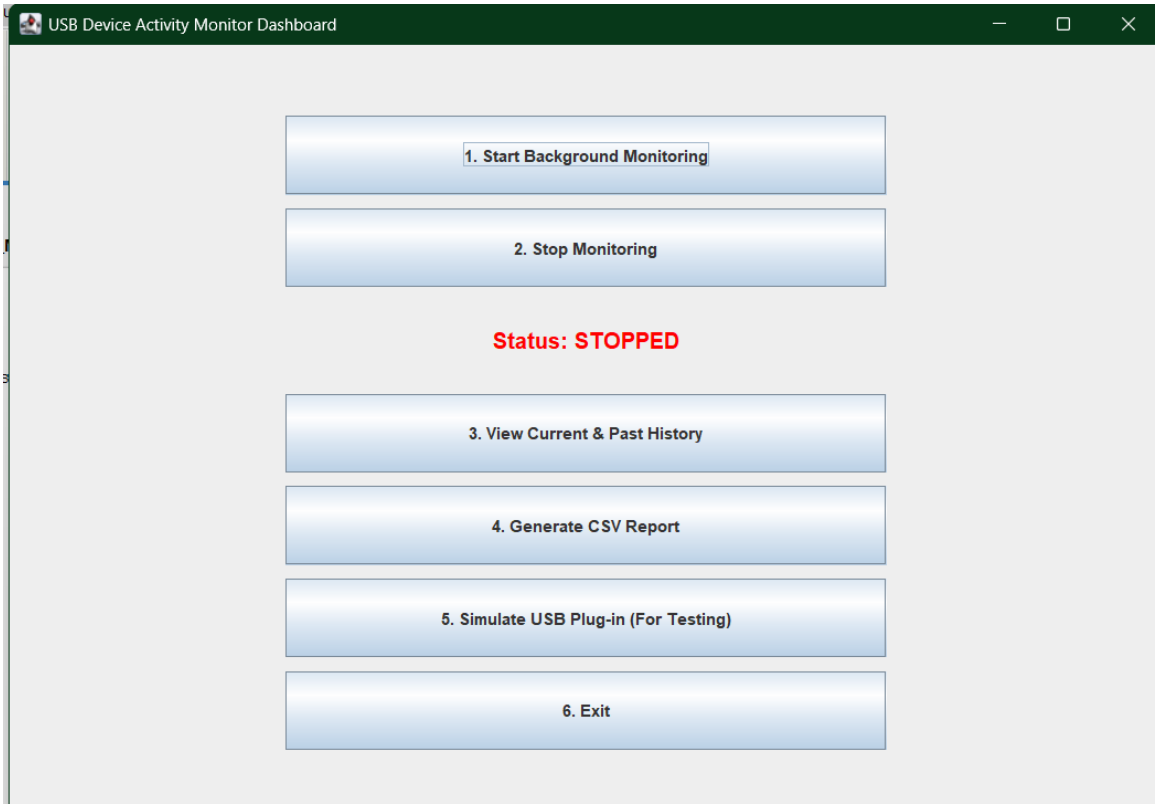


Figure 1: Main dashboard with start and stop monitoring, history, CSV report, simulated plug-in and exit; the status label shows STOPPED before monitoring starts.

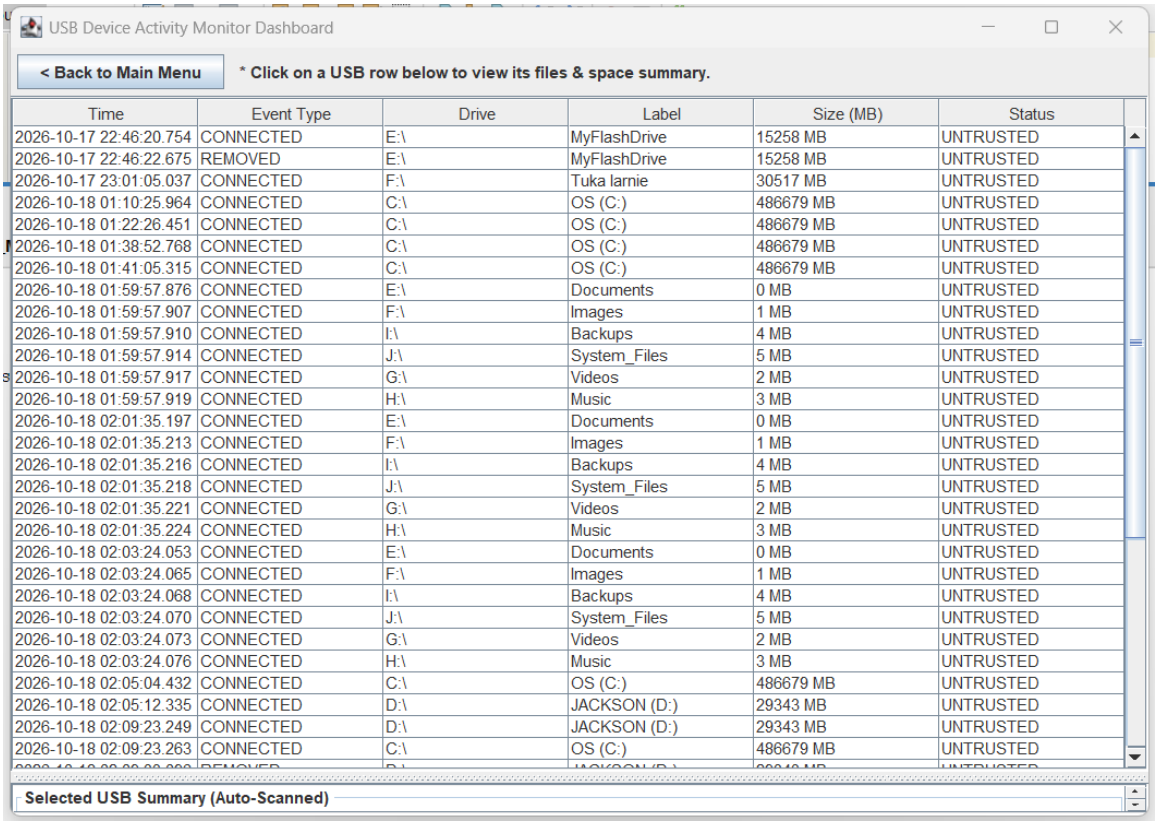
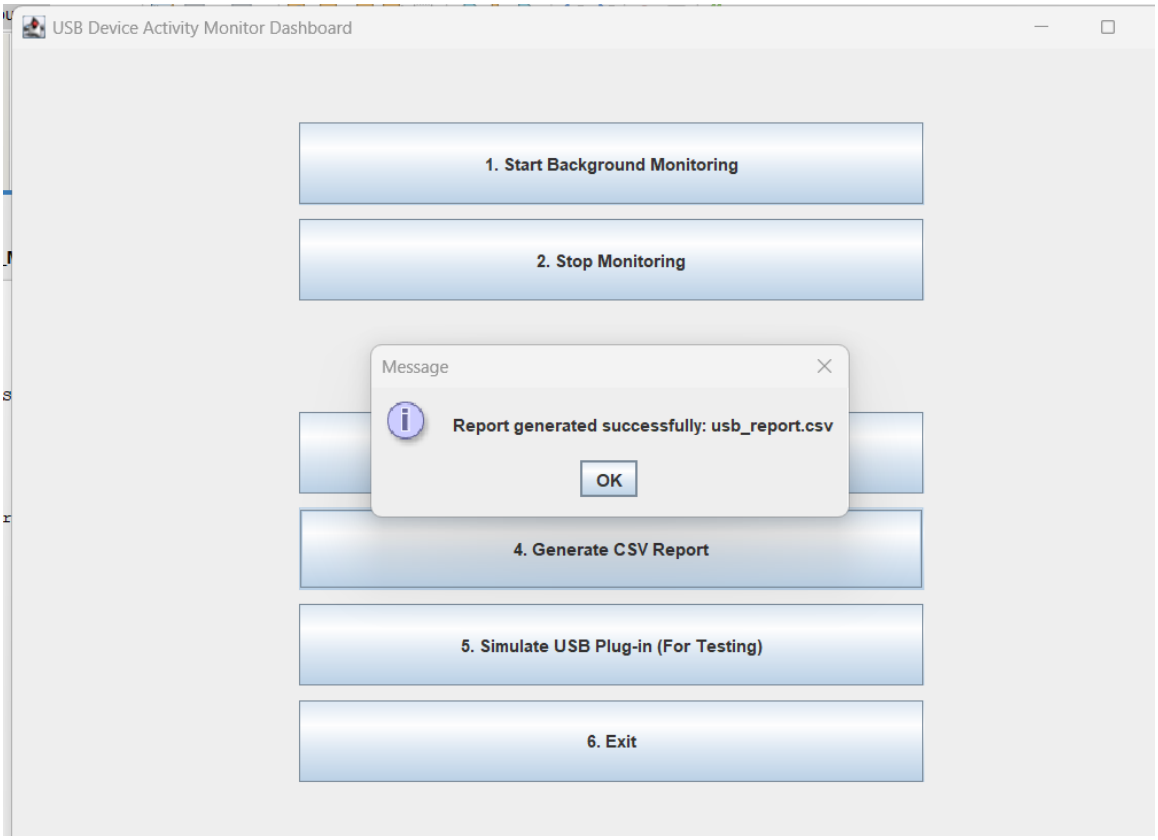


Figure 2: History view with connected and removed events, drive, label, size (MB) and trusted status. Clicking a row shows that drive's files and space summary.


USB Device Activity Monitor Dashboard
1. Start Background Monitoring
2. Stop Monitoring
Message
Report generated successfully: usb_report.csv
OK
4. Generate CSV Report
5. Simulate USB Plug-in (For Testing)
6. Exit

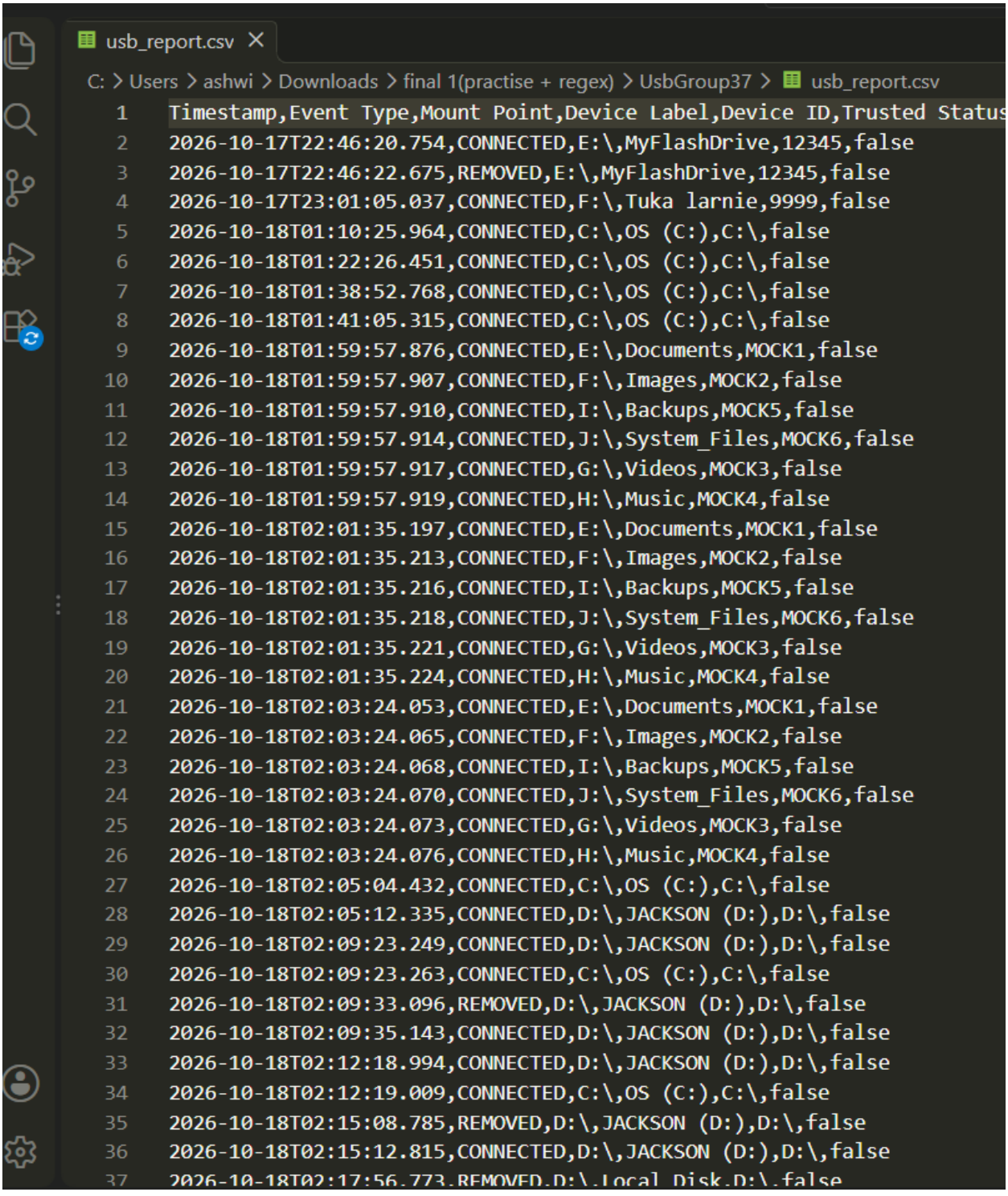


Figure 3: CSV export from the GUI, confirming that usb_report.csv was generated.

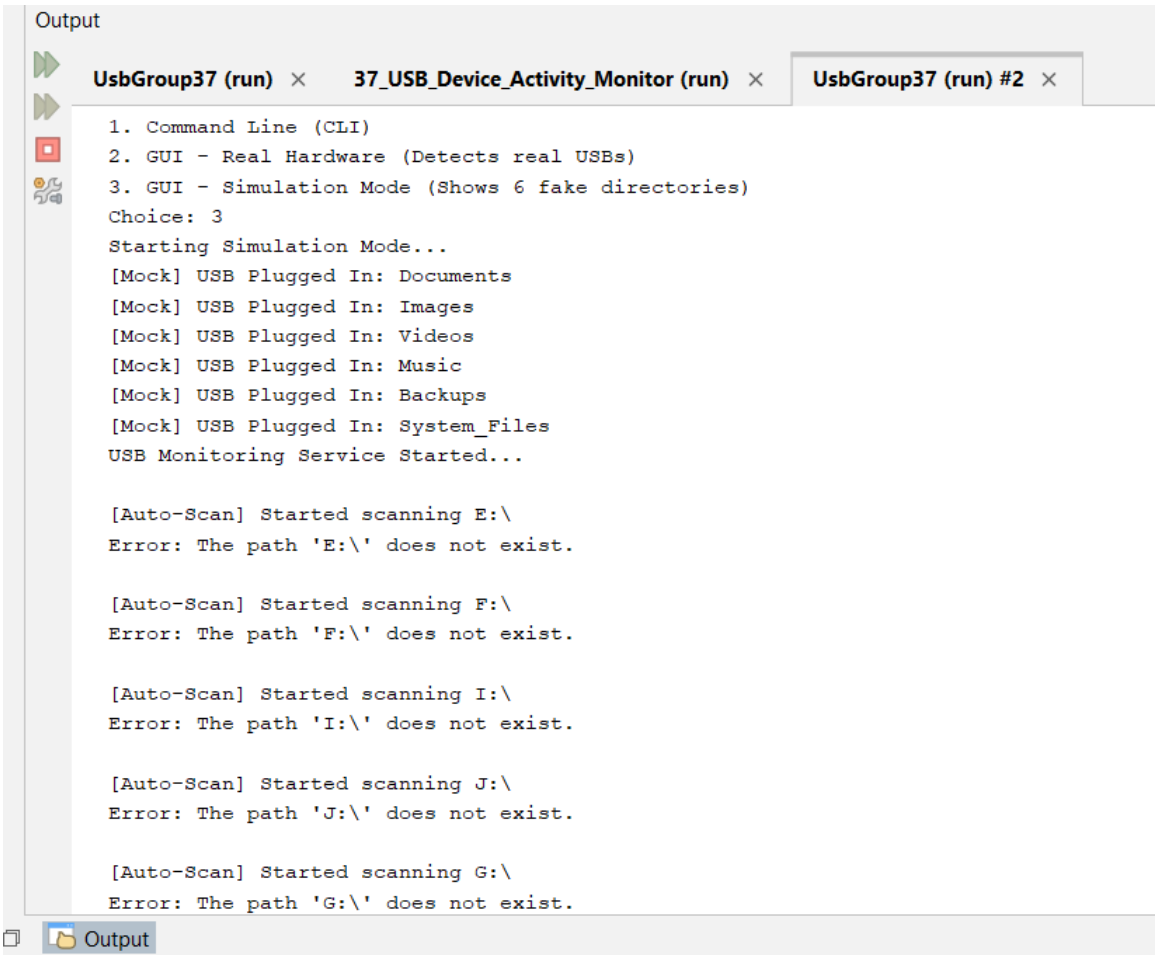


Figure 4: NetBeans console in Simulation Mode: six mock drives are plugged in, monitoring starts, and the auto-scan reports a missing path as an error message instead of crashing (captured before mock drives were given simulated scan results).

5.4 Evaluation

- **Correctness:** connection, removal, history persistence, CSV export and error handling worked in the tests above.
- **Efficiency:** detection latency is at most one polling interval (2 seconds). Trust checks are O(1), search is O(n), sort is O(n log n) and the scan is linear in the number of files and folders.
- **Ease of use:** the dashboard has six clearly labelled buttons, and the history table can be searched and sorted by time from the same screen.

5.5 Known limitations

- **Fixed drives can appear as USB events:** the detector skips local, fixed and network drives by matching the Windows drive-type description, which is not

reliable on every machine. In testing, the history contained CONNECTED events for the system drive C:\.

- **Simulation Mode auto-scan:** in an earlier build, mock drives (for example E:\) were scanned as real paths and reported as missing (Figure 4). The current version returns simulated scan results for any drive whose ID starts with MOCK, so mock drives no longer trigger a real scan.
- **Weak device identification:** Java has no native USB API, so devices are identified by drive-level information (label, mount point, size, file system). Two identical drives could be confused.
- **Polling delay:** an event is detected only at the next polling cycle.
- **No revoke and limited CLI:** a device can be marked as trusted in the GUI but cannot be un-trusted, and the CLI has no search, sort or trust options.
- **Trust status in reports:** the CSV records the trust status at the time of the event, while the GUI table shows the device's current status.

6. Challenges Faced

Challenge	Type	What we did
No native Java USB API	Technical	Used polling of drive roots and compared drive sets between polls
Weak device identification	Technical	Built the device ID from drive attributes and documented the limitation
Testing without hardware on every machine	Technical	Built MockUsbDetector and Simulation Mode
Fixed drives appearing as USB events	Technical	Added a filter on the Windows drive-type description; it is not reliable on every machine, so it is documented as a known limitation
Keeping the UML and code in sync	Teamwork	Renamed UsbDevice to UsbDrive, simplified the Observer so only the GUI registers as a listener, and redrew the UML from the final code
Mock drives triggering real folder scans	Technical	Gave simulated scan results to drives whose ID starts with MOCK

7. Graduate Attribute Reflection

7.1 GA2 – Application of Scientific and Engineering Knowledge

We applied the Java and software engineering concepts from SDN260S to a real security problem. Object-oriented design with interfaces and polymorphism made the

system modular and extendable. Generic collections (List, Set, Map) were chosen for their access complexity: a HashSet for trusted IDs gives constant-time lookups. Recursion powers the DriverScanner, lambdas, streams and Comparator implement search and sort, serialization persists history, file handling produces the log and CSV report, and a ScheduledExecutorService provides background detection. We followed an engineering process of specifying the problem, designing with UML, implementing, and verifying with normal and edge-case tests.

7.2 GA7 – Sustainability and Impact of Engineering Activity

Polling at a fixed interval on a background thread avoids a busy loop, which keeps CPU use and energy consumption low and lets the monitor run continuously. History is held in memory and saved to a compact serialized file, so it is read from disk only once at startup. Modular code, interfaces and a mock detector make the software easy to maintain and extend, which reduces future rework. The wider impact is better data protection: by recording and flagging untrusted USB devices, the tool helps individuals and small organisations reduce data theft, malware and accidental data loss, without needing costly commercial software. Its limits, namely polling delay and weak device identification, are stated in Section 5.5.

8. Team Contributions

Member	Role	Classes / tasks
Thato Ernest	Team Lead, Tester / QA	UsbGroup37, testing, report, integration
Tumisang Mainaakae	Core Service and UI	UsbMonitorService, PollingUsbDetector, CliMenu, MonitorGui
Ashwill Visagie	Data & Storage, Documentation	UsbDrive, UsbEvent, EventType, EventRepository, TrustedDeviceRegistry, UML, README, report
Kwalunga Ligwa	Logging & Reports	EventLogger, DriverScanner, ReportGenerator
Wande Mchwayi	Documentation	Report sections 1 and 4

9. Conclusion and Future Work

The USB Device Activity Monitor meets its core goals: it detects when USB drives are connected and removed, logs and persists events, flags untrusted devices, scans drives recursively, exports CSV reports and offers both a CLI and a GUI. Simulation Mode allows testing without hardware. Future work: exclude fixed drives reliably, add a revoke option for trusted devices, add search, sort and trust options to the CLI, use operating-

system events instead of polling, and identify devices by hardware vendor and serial number.